AERO ADMS

Session 2024 – 2028

Submitted By:

Muhammad Umer	2024-CS-191
Khawaja Bilal Ahmad	2024-CS-199
Muhammad Saad Asif	2024-CS-200
Rustgaar Ahmad	2024-CS-234

Supervised By:

Mr. Waleed

Course:

CSC201L – Advanced Database Management Systems Lab

Department of Computer Science

University of Engineering and Technology, Lahore

Contents

1. Introduction 3
1.1. Project Vision and Purpose 3
1.2. System Scope 3
1.3. Technology Stack 3
2. High-Level System Architecture 4
2.1. Polyglot Persistence Strategy 4
2.2. Containerization & Infrastructure 4
2.3. Server Lifecycle & Autopilot 4
3. Database Design & Engineering 5
3.1. The Relational Tier (PostgreSQL) 5
3.2. The Graph Engine (Neo4j) 6
3.3. The Real-Time Telemetry Tier (MongoDB) 6
3.4. The Speed & Memory Layer (Redis) 6
4. Core Backend Functionalities 7
4.1. Transactional Booking Engine 7
4.2. Distributed Seat Locking (Redis) 7
4.3. Advanced Flight Routing (Neo4j) 8
4.4. Dynamic Graph Pruning 9
4.5. Live Flight Tracker (WebSockets & MongoDB) 9
4.6. Flight Search Caching 9
5. Frontend User Interface 10
6. Testing, Validation & Error Handling 12
6.1. Concurrency Testing (The Redis Lock) 13
6.2. System Self-Healing (Graph Restoration) 13
7. Conclusion & Future Enhancements 13
7.1. Project Summary 13
7.2. Future Roadmap 14

Figures:

Figure 1: Customer Dashboard 11
Figure 2: Flight Booking Page 11
Figure 3: Analytics Page 12
Figure 4: Admin Page 12

1. Introduction

1.1. Project Vision and Purpose

The AERO Airline Database Management System (ADMS) is a highly scalable, distributed backend infrastructure designed to solve the complexities of modern aviation data management. Traditional monolithic databases struggle to balance strict transactional integrity (like preventing double-booked seats) with the need for high-speed, flexible analytics (like route optimization and live telemetry). AERO ADMS bridges this gap by implementing a polyglot microservice architecture, ensuring data consistency, real-time tracking, and optimal routing for a seamless passenger experience.

1.2. System Scope

The core capabilities of the AERO system encompass four distinct operational domains:

- **Distributed Flight Scheduling:** Managing fleets, airports, and flight timelines across geographically distinct database shards.
- **Transactional Booking Engine:** Handling passenger seat reservations with strict concurrency controls and atomic rollbacks.
- **Intelligent Route Optimization:** Autonomously calculating the cheapest and fastest multi-hop layover connections.
- **Live Flight Telemetry:** Streaming real-time aircraft coordinates to client dashboards using non-blocking WebSockets.

1.3. Technology Stack

To achieve this, the system leverages a modern, containerized stack:

- **Backend Framework:** FastAPI (Python)
- **Frontend UI:** React.js
- **Relational Database:** PostgreSQL (Geographically Sharded)
- **Graph Database:** Neo4j
- **Document Database:** MongoDB
- **In-Memory Store:** Redis
- **Infrastructure:** Docker & Docker Compose

1.4. Target Audience & Stakeholders

AERO ADMS is designed to serve multiple tiers of users simultaneously:

- **Passengers:** Benefit from instantaneous flight searches, guaranteed seat reservations without double-booking errors, and live tracking of their scheduled aircraft.
- **System Administrators**: Gain access to an automated, self-healing database infrastructure that autonomously manages graph pruning and handles regional data safely across distributed shards.

2. High-Level System Architecture

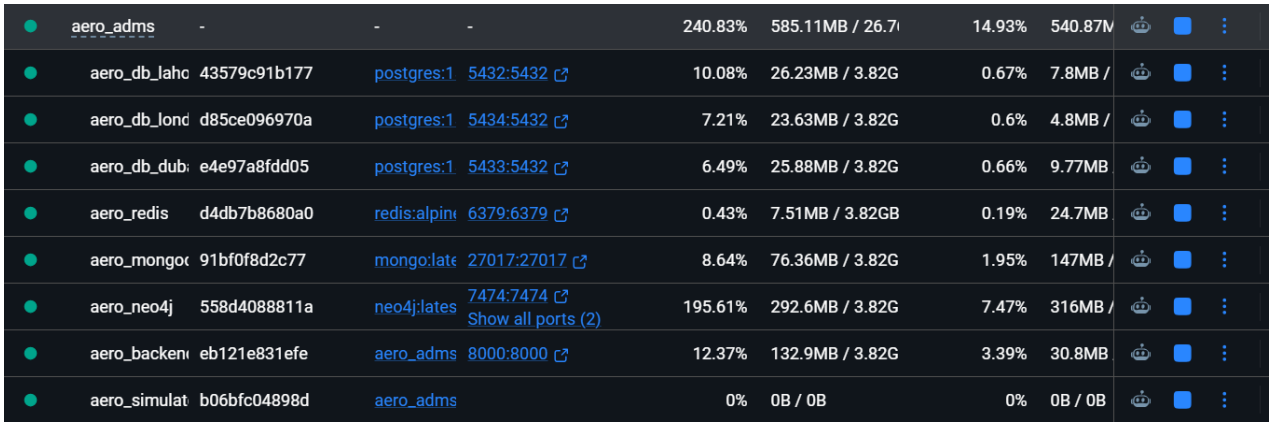
2.1. Polyglot Persistence Strategy

The defining characteristic of AERO ADMS is its "Polyglot Persistence" model—the practice of using different database engines to handle specific, optimized workloads rather than forcing a single database to do everything poorly.

- **PostgreSQL (The Vault):** Serves as the source of truth for critical ACID transactions (bookings, payments, passenger data). The data is partitioned into regional shards (e.g., Lahore, Dubai, London) to localize query execution and prevent a single point of failure.
- **Neo4j (The Navigator):** Treats airports as nodes and flights as edges, allowing the system to execute complex routing algorithms (shortest path, cost reduction) in milliseconds without joining massive SQL tables.
- **MongoDB (The Tracker):** Handles the high-throughput, schema-less ingestion of live aircraft coordinates, decoupling volatile telemetry data from the strict relational core.
- **Redis (The Accelerator):** Operates as an in-memory caching layer to serve frequent routing queries instantly and manages distributed locks to prevent race conditions during seat checkouts.

2.2. Containerization & Infrastructure

The entire architecture is containerized using Docker. A docker-compose.yml network binds the FastAPI application and the diverse database engines into an isolated virtual grid. This ensures absolute environment parity, allowing the system to be spun up, synchronized, and tested predictably without host-machine conflicts.



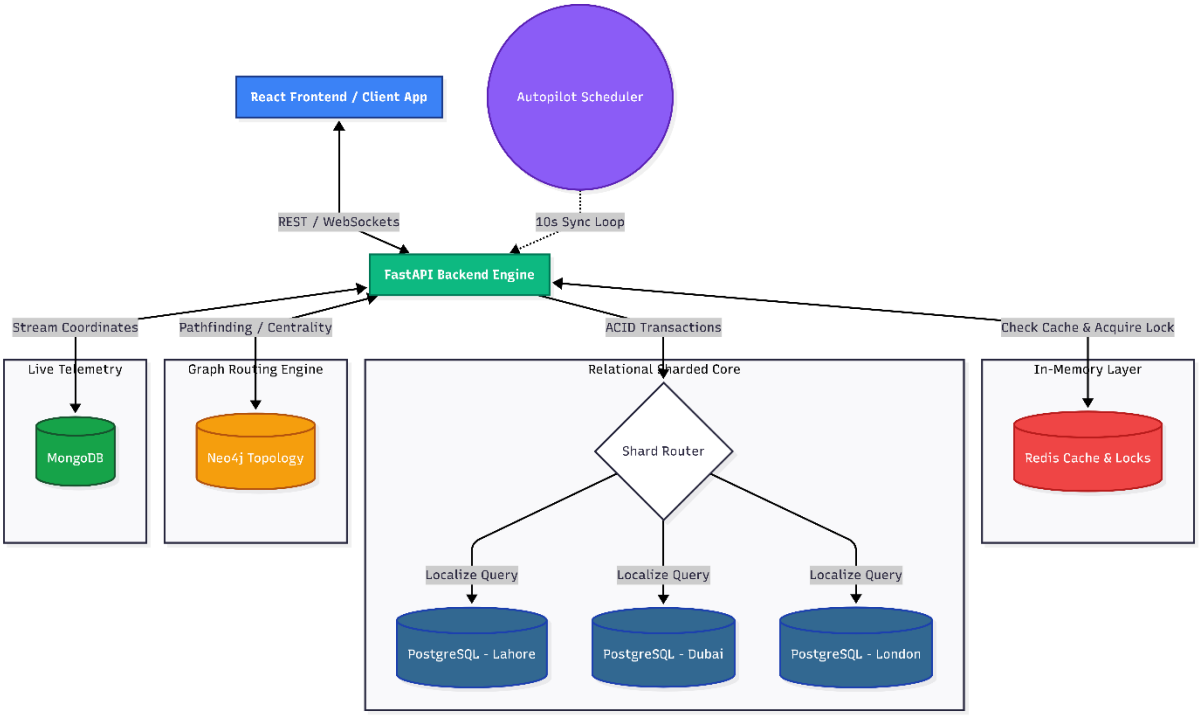
aero_adms	-	-	-	240.83%	585.11MB / 26.7	14.93%	540.87M	
aero_db_lahc	43579c91b177	postgres:1	5432:5432	10.08%	26.23MB / 3.82G	0.67%	7.8MB /	
aero_db_lond	d85ce096970a	postgres:1	5434:5432	7.21%	23.63MB / 3.82G	0.6%	4.8MB /	
aero_db_dub	e4e97a8fdd05	postgres:1	5433:5432	6.49%	25.88MB / 3.82G	0.66%	9.77MB	
aero_redis	d4db7b8680a0	redis:alpin	6379:6379	0.43%	7.51MB / 3.82GB	0.19%	24.7MB	
aero_mongo	91bf0f8d2c77	mongo:late	27017:27017	8.64%	76.36MB / 3.82G	1.95%	147MB /	
aero_neo4j	558d4088811a	neo4j:lates	7474:7474 Show all ports (2)	195.61%	292.6MB / 3.82G	7.47%	316MB /	
aero_backen	eb121e831efe	aero_adms	8000:8000	12.37%	132.9MB / 3.82G	3.39%	30.8MB	
aero_simulat	b06bfc04898d	aero_adms		0%	0B / 0B	0%	0B / 0B	

2.3. Server Lifecycle & Autopilot

The application relies on FastAPI lifespan events to trigger self-healing synchronization upon server startup. Before accepting HTTP traffic, the backend autonomously reads the sharded PostgreSQL tables and mirrors the network topology into Neo4j. Furthermore, a background worker (APScheduler) operates on a 10-

second loop, continually scanning for scheduled flights, triggering live telemetry in MongoDB, and pruning fully booked edges from the graph network.

Architecture Diagram



.

3. Database Design & Engineering

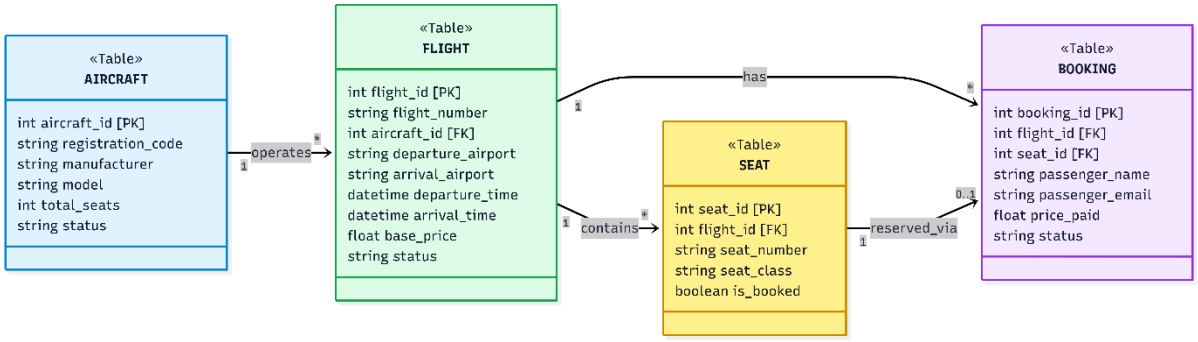
Because AERO ADMS handles everything from strict financial transactions to high-speed geographical routing, a single database paradigm would create severe bottlenecks. The data tier is split into four specialized engines.

3.1. The Relational Tier (PostgreSQL)

PostgreSQL acts as the foundational "source of truth." It is strictly responsible for persistent, ACID-compliant data where consistency is non-negotiable—such as aircraft inventory, flight schedules, and passenger bookings.

To ensure high availability and low latency, the PostgreSQL layer is geographically sharded. When a flight is created, the system inspects the departure_airport and routes the record to the appropriate regional shard (e.g., Lahore, Dubai, or London). This ensures that heavy booking traffic in the Middle East does not degrade database performance for European flights.

Entity-Relationship Diagram



3.2. The Graph Engine (Neo4j)

While PostgreSQL handles the bookings, it is highly inefficient for calculating multi-hop layovers. To solve this, the network topology is mirrored into Neo4j.

- **Nodes:** Represent Airport entities, uniquely identified by their IATA codes (e.g., LHE, DXB).
- **Edges:** Represent directed FLIGHT relationships connecting airports. Each edge stores critical weights, including base_price, departure_time, and arrival_time. This structure allows the backend to execute recursive Cypher queries to traverse the graph and calculate the cheapest or fastest routes in milliseconds.

3.3. The Real-Time Telemetry Tier (MongoDB)

Tracking active flights generates a massive volume of volatile, unstructured data that would overwhelm the PostgreSQL transaction logs. MongoDB is utilized to ingest this telemetry.

- **Document Structure:** Active flights are stored in the live_flights collection as JSON documents containing the flight number, status, and nested geolocation coordinates (latitude/longitude). This allows the frontend to poll or stream active flight positions without ever touching the core booking databases.

3.4. The Speed & Memory Layer (Redis)

To protect the disk-based databases from redundant queries and race conditions, Redis is deployed as an in-memory acceleration and security layer.

- **Search Caching:** Complex Neo4j routing queries are serialized and stored as key-value pairs (e.g., cache:cheapest:LHE:DXB) with a 60-second Time-To-Live (TTL).
- **Distributed Locking:** Temporary keys (e.g., lock:seat:999:9991) are generated when a user selects a seat, locking out other users for exactly 10 minutes to guarantee a conflict-free checkout window.

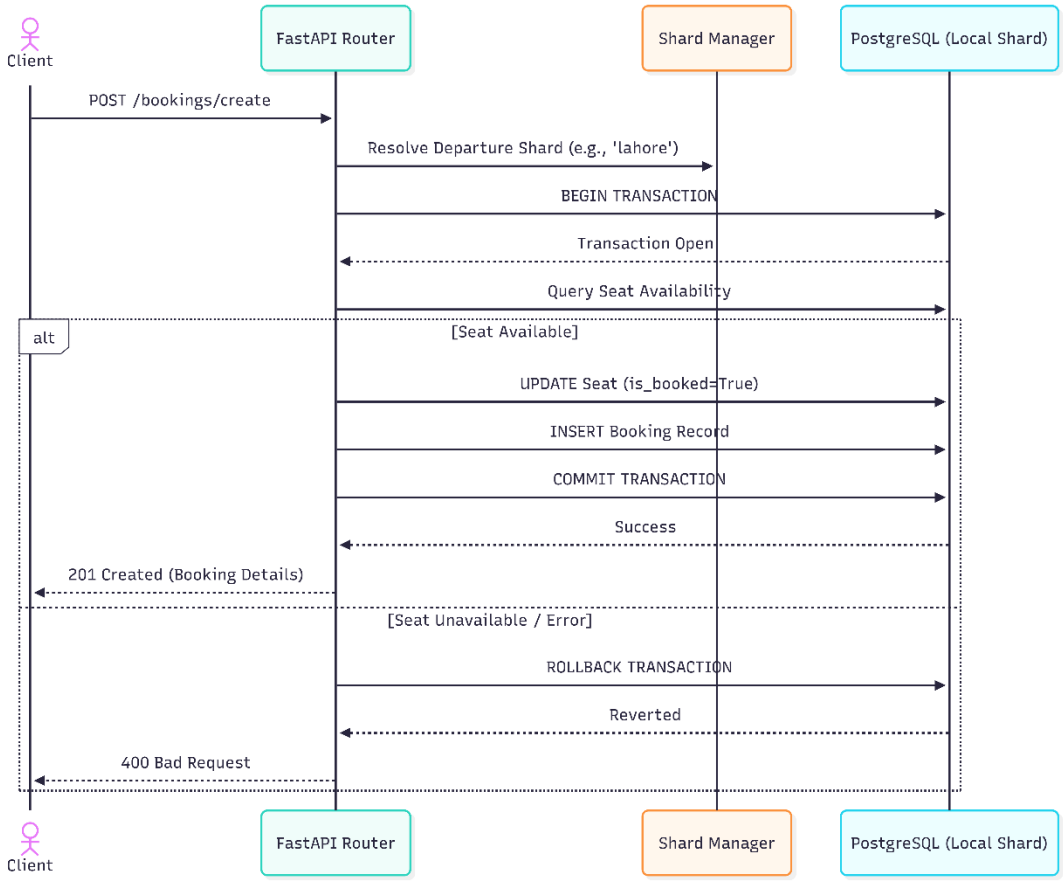
4. Core Backend Functionalities

The AERO ADMS backend is designed to handle high concurrency, ensure strict data integrity, and provide low-latency responses. This is achieved through a combination of relational transactions, distributed memory locks, and graph algorithms.

4.1. Transactional Booking Engine

The booking engine operates within strict ACID (Atomicity, Consistency, Isolation, Durability) boundaries managed by PostgreSQL and SQLAlchemy context managers. When a passenger submits a booking, the system identifies the correct geographical shard based on the departure airport. It opens an atomic transaction, validates seat availability, calculates pricing, and executes the booking. If any step fails (e.g., payment failure or constraint violation), the context manager triggers an automatic rollback, ensuring no partial data is ever saved to the database.

Sequence Diagram

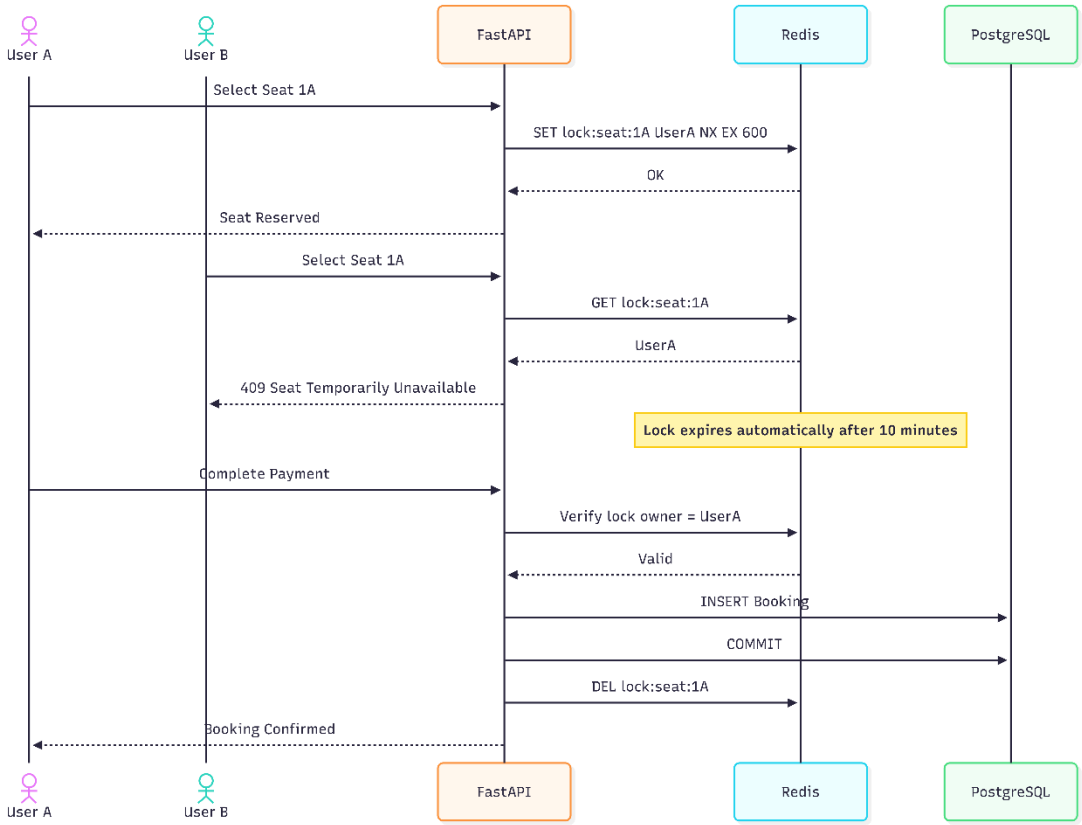


4.2. Distributed Seat Locking (Redis)

To solve the classic "Double Booking" race condition—where two users attempt to purchase the same seat simultaneously—the system utilizes a distributed locking mechanism via Redis. When a user clicks a seat, the system writes a temporary key (e.g., lock:seat:999:9991) mapped to their email with a 600-second Time-To-Live

(TTL). If a second user attempts to check out with that seat, the API checks Redis first. Recognizing the lock, it immediately rejects the second user with a 409 Conflict before ever touching the PostgreSQL database, preserving database bandwidth. Upon successful checkout, the lock is manually deleted.

Sequence Diagram

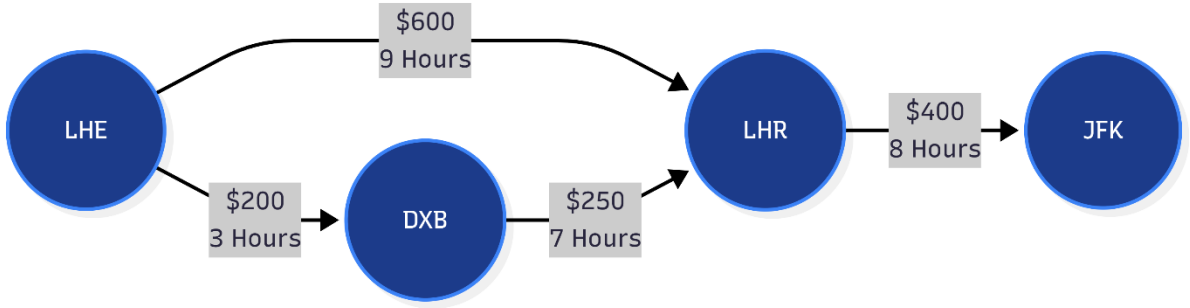


4.3. Advanced Flight Routing (Neo4j)

AERO ADMS eliminates the need for expensive SQL JOIN operations when searching for multi-hop flights by delegating routing to Neo4j. The Graph database utilizes native algorithms to calculate ideal paths across the network topology:

- **Cheapest Path:** A Cypher query utilizes the reduce() function to sum the base_price of all edges (flights) along a path up to 3 hops away, ordering the results by minimum cost.
- **Fastest Path:** The algorithm calculates the chronological duration between the first departure and final arrival, strictly enforcing chronological layover validity (the arrival time of leg 1 must precede the departure time of leg 2).

Graph Diagram



4.4. Dynamic Graph Pruning

To maintain perfect synchronization between the SQL inventory and the Neo4j routing engine, the system implements a dynamic graph pruner. Triggered autonomously after a booking transaction or cancellation, the pruner queries PostgreSQL to check the total seat capacity versus booked seats for a specific flight. If the flight reaches 100% capacity, the utility actively deletes that specific flight edge from Neo4j. This guarantees that fully booked flights never appear in customer search results, creating a self-healing data pipeline.

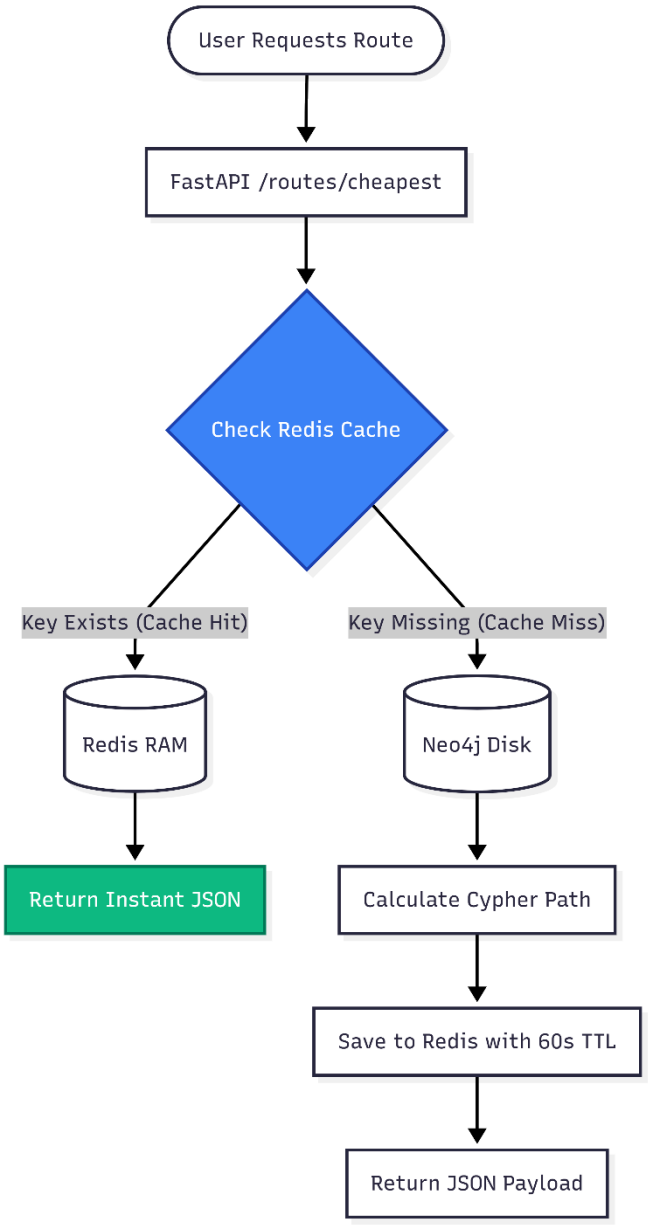
4.5. Live Flight Tracker (WebSockets & MongoDB)

Real-time telemetry is decoupled entirely from the core booking engine to prevent performance degradation. An asynchronous background scheduler (APScheduler) scans PostgreSQL every 10 seconds for flights past their departure time. It marks them as 'Completed' in SQL, deletes their routing edge in Neo4j, and injects a live tracking document into MongoDB. The FastAPI application then utilizes WebSockets to continuously poll MongoDB and push coordinate updates directly to the connected React client interface.

4.6. Flight Search Caching

Because Neo4j recursive pathfinding is computationally expensive, Redis is deployed as a caching layer to optimize read-heavy routing endpoints. When a user queries a route (e.g., LHE to DXB), the API generates a unique key (cache:cheapest:LHE:DXB). If the key exists in RAM, the pre-calculated path is returned instantly (Cache Hit). If missing (Cache Miss), the API queries Neo4j, returns the result to the user, and simultaneously saves the JSON payload to Redis with a 60-second expiration window to serve subsequent users instantaneously.

Flowchart Diagram



5. Frontend User Interface

The AERO ADMS frontend is a modern, single-page application (SPA) built with React.js. It is designed to consume the diverse APIs provided by the FastAPI backend, translating complex graph routes, live WebSocket streams, and transactional databases into a clean, intuitive, and highly responsive customer dashboard.

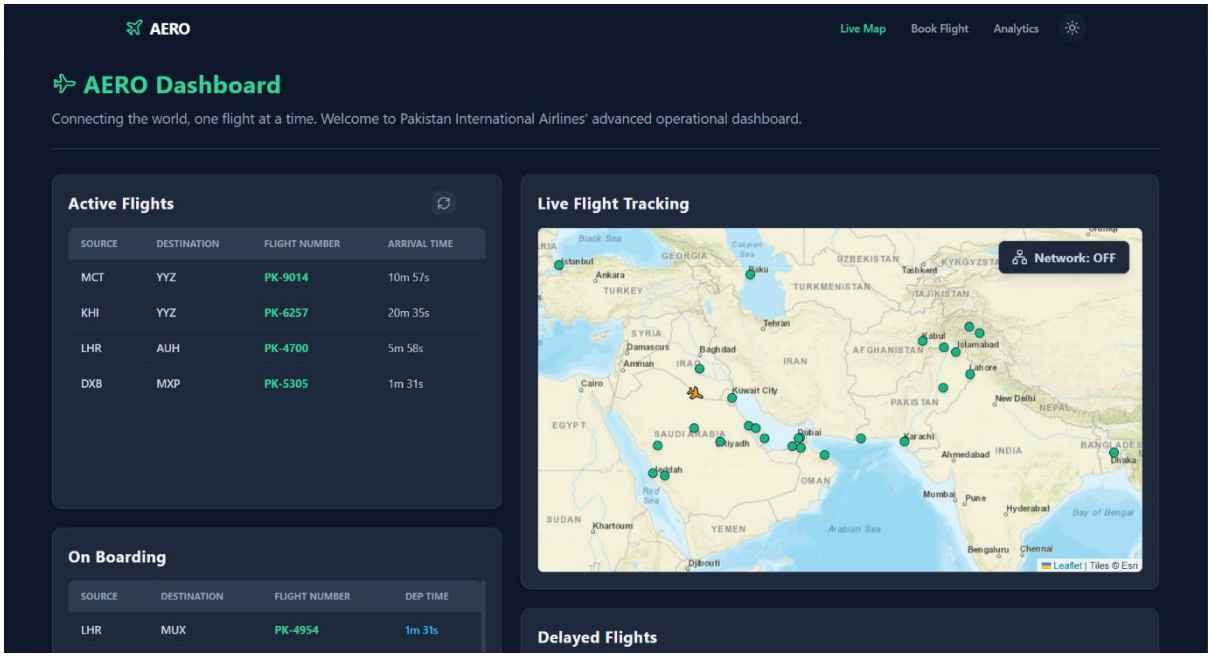


Figure 1: Customer Dashboard

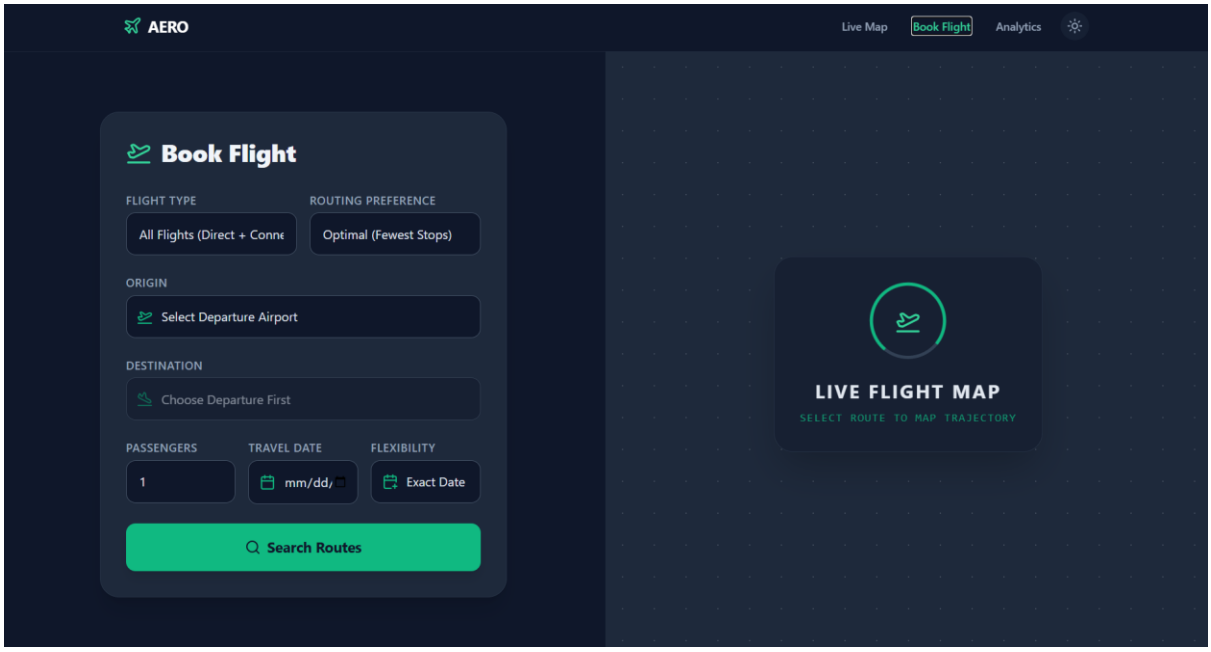


Figure 2: Flight Booking Page

Figure 3: Analytics Page

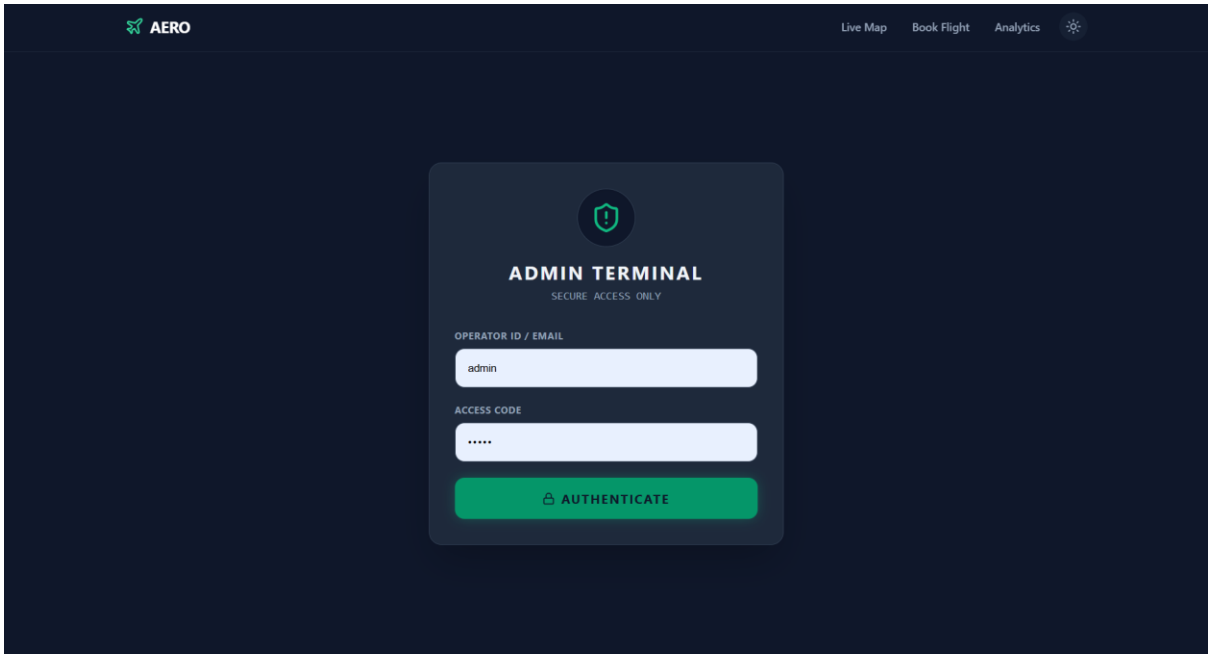


Figure 4: Admin Page

6. Testing, Validation & Error Handling

To ensure AERO ADMS is enterprise-ready, the system was subjected to targeted validation tests, focusing on high-concurrency conflict resolution and autonomous data recovery

6.1. Concurrency Testing (The Redis Lock)

1. **The Setup:** A single seat (Seat 1A) on a specific flight was identified.
2. **The Execution:** Two separate HTTP POST requests (simulating User A and User B) were fired simultaneously at the /bookings/create endpoint targeting the exact same seat.
3. **The Result:** The Redis in-memory engine successfully processed User A's request first, applying the 10-minute TTL lock in under a millisecond. User B's request was intercepted by the lock validation block and instantly rejected with a 409 Conflict: Transaction Denied error.
4. **Validation:** The PostgreSQL database was queried, confirming that only one booking record was created and the seat was cleanly marked as is_booked = True.

6.2. System Self-Healing (Graph Restoration)

1. The system executes a PostgreSQL transaction to free the seat (is_booked = False) and refund the booking.
2. The update_graph_flight_capacity utility is triggered.
3. It detects that the total booked seats are now strictly less than the aircraft's total capacity.
4. It executes a Cypher MERGE command in Neo4j, which safely and idempotently restores the flight edge between the origin and destination airports, instantly making the flight available for new route searches.

7. Conclusion & Future Enhancements

7.1. Project Summary

The AERO Airline Database Management System successfully demonstrates the immense power of Polyglot Persistence in modern microservice architectures. By refusing to compromise on database selection, the system achieves the impossible: strict financial ACID compliance (PostgreSQL), lightning-fast multi-hop layover calculations (Neo4j), real-time non-blocking telemetry (MongoDB), and atomic race-condition prevention (Redis).

Containerized via Docker, the backend operates as a self-healing, self-managing engine, providing the React frontend with robust, high-speed data feeds. AERO ADMS stands as a production-grade blueprint for scalable aviation data management.

7.2. Future Roadmap

While the core architecture is complete and highly optimized, future iterations of the platform could implement the following enhancements:

- **JWT Authentication:** Implementing JSON Web Tokens (JWT) and OAuth2 to secure passenger accounts and provide role-based access control (RBAC) for airline administrators.
- **Payment Gateway Integration:** Connecting the transactional core to a service like Stripe to process real-world credit card holds during the 10-minute Redis lock window.
- **Kubernetes Orchestration:** Migrating from Docker Compose to a managed Kubernetes cluster (e.g., AWS EKS or Google GKE) to allow individual database shards to auto-scale horizontally based on regional API traffic.